

ABSTRACT CLASS IN DART

Introduction

In this section, you will learn about **dart abstract class**. Before learning about abstract class, you should have a basic understanding of [class](#), [object](#), [constructor](#), and [inheritance](#). Previously you learned how to define a class. These classes are **concrete classes**. You can create an object of concrete classes, but you cannot create an object of abstract classes.

Abstract Class

Abstract classes are classes that cannot be initialized. It is used to define the behavior of a class that can be inherited by other classes. An abstract class is declared using the keyword **abstract**.

Syntax

```
abstract class ClassName {  
    //Body of abstract class  
  
    method1();  
    method2();  
}
```

Abstract Method

An abstract method is a method that is declared without an implementation. It is declared with a semicolon (;) instead of a method body.

Syntax

```
abstract class ClassName {
```

```
//Body of abstract class
method1();
method2();
}
```

Why We Need Abstract Class

Subclasses of an abstract class must implement all the abstract methods of the abstract class. It is used to achieve abstraction in the Dart programming language.

Example 1: Abstract Class In Dart

In this example below, there is an abstract class **Vehicle** with two abstract methods **start()** and **stop()**. The subclasses **Car** and **Bike** implement the abstract methods and override them to print the message.

```
abstract class Vehicle {
  // Abstract method
  void start();
  // Abstract method
  void stop();
}

class Car extends Vehicle {
  // Implementation of start()
  @override
  void start() {
    print('Car started');
  }

  // Implementation of stop()
  @override
  void stop() {
    print('Car stopped');
  }
}

class Bike extends Vehicle {
  // Implementation of start()
  @override
  void start() {
    print('Bike started');
  }

  // Implementation of stop()
  @override
  void stop() {
    print('Bike stopped');
  }
}
```

```

    }
}

void main() {
    Car car = Car();
    car.start();
    car.stop();

    Bike bike = Bike();
    bike.start();
    bike.stop();
}

```

[Show Output](#)

Run Online

Info

Note: The abstract class is used to define the behavior of a class that can be inherited by other classes. You can define an abstract method inside an abstract class.

Example 2: Abstract Class In Dart

In this example below, there is an abstract class **Shape** with one abstract method **area()** and two subclasses **Rectangle** and **Triangle**. The subclasses implement the **area()** method and override it to calculate the area of the rectangle and triangle, respectively.

```

abstract class Shape {
    int dim1, dim2;
    // Constructor
    Shape(this.dim1, this.dim2);
    // Abstract method
    void area();
}

class Rectangle extends Shape {
    // Constructor
    Rectangle(int dim1, int dim2) : super(dim1, dim2);

    // Implementation of area()
    @override
    void area() {
        print('The area of the rectangle is ${dim1 * dim2}');
    }
}

```

```

class Triangle extends Shape {
    // Constructor
    Triangle(int dim1, int dim2) : super(dim1, dim2);

    // Implementation of area()
    @Override
    void area() {
        print('The area of the triangle is ${0.5 * dim1 * dim2}');
    }
}

void main() {
    Rectangle rectangle = Rectangle(10, 20);
    rectangle.area();

    Triangle triangle = Triangle(10, 20);
    triangle.area();
}

```

[Show Output](#)

[Run Online](#)

Constructor In Abstract Class

You can't create an object of an abstract class. However, you can define a constructor in an abstract class. The constructor of an abstract class is called when an object of a subclass is created.

Example 3: Constructor In Abstract Class

In this example below, there is an abstract class **Bank** with a constructor which takes two parameters **name** and **rate**. There is an abstract method **interest()**. The subclasses **SBI** and **ICICI** implement the abstract method and override it to print the interest rate.

```

abstract class Bank {
    String name;
    double rate;

    // Constructor
    Bank(this.name, this.rate);

    // Abstract method
    void interest();

    //Non-Abstract method: It have an implementation
    void display() {

```



```

        print('Bank Name: $name');
    }
}

class SBI extends Bank {
    // Constructor
    SBI(String name, double rate) : super(name, rate);

    // Implementation of interest()
    @override
    void interest() {
        print('The rate of interest of SBI is $rate');
    }
}

class ICICI extends Bank {
    // Constructor
    ICICI(String name, double rate) : super(name, rate);

    // Implementation of interest()
    @override
    void interest() {
        print('The rate of interest of ICICI is $rate');
    }
}

void main() {
    SBI sbi = SBI('SBI', 8.4);
    ICICI icici = ICICI('ICICI', 7.3);

    sbi.interest();
    icici.interest();
    icici.display();
}

```

[➤ Show Output](#)

Run Online

Key Points To Remember

- You can't create an object of an abstract class.
- It can have both abstract and non-abstract methods.
- It is used to define the behavior of a class that other classes can inherit.
- Abstract method only has a signature and no implementation.

Video

Watch our video on abstract class in Dart.